

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Academic tutoring has traditionally depended on physical proximity between a tutor and a student: a fixed classroom, a scheduled time, and a shared physical space. Within a typical Nigerian tertiary institution such as the Federal Polytechnic, Ilaro, students who require extra academic support outside regular lecture hours often rely on informal arrangements locating a willing course representative, a senior colleague, or a lecturer, and physically coordinating a time and venue to meet. This arrangement is frequently unreliable: it depends on chance encounters, verbal agreements that are easily forgotten, and the availability of a suitable physical location, none of which scale well as student populations grow.

The emergence of electronic learning (e-learning) has offered an alternative. E-learning refers broadly to the use of digital technologies and the internet to support and deliver teaching and learning, allowing learners to access educational content and interact with instructors from any location and at flexible times (Montenegro Rueda et al., 2024). Contemporary e-learning platforms increasingly combine several previously separate tools content delivery, scheduling, and live video communication into a single coherent system, reducing the friction that discourages students from seeking help and tutors from offering it.

Within Nigerian tertiary education specifically, the adoption of information and communication technology (ICT) to support teaching and learning remains an ongoing process, shaped by infrastructural, financial, and attitudinal factors (Obisanya, Aminu, & Adebola, 2025). A study of Federal Polytechnic Ilaro found that while there is a clear institutional appetite for digital tools to improve service delivery, adoption is hindered by resistance to change and gaps in relevant digital skills among staff, and recommended structured exposure to digital innovation to address this (Obisanya, Aminu, & Adebola, 2025). This pattern is consistent with the broader theory of diffusion of innovations, which holds that the rate at which a new tool is adopted within an organisation depends less on the tool's technical sophistication than on how compatible it is with users' existing habits and how easily its benefits can be observed firsthand (Rogers, 2003). This creates a practical opportunity: a purpose-built, department-specific system that is simple enough for a general student and tutor population to adopt without extensive training.

This project responds directly to that opportunity. It presents the design and implementation of a Tutor and Student E-Learning Meeting System developed specifically for the Department of

Computer Sciences, Federal Polytechnic, Ilaro. The system allows registered students to discover departmental tutors by subject and specialization, request and manage tutoring sessions, and join live one-on-one video sessions directly from a web browser, while allowing tutors to manage their availability, confirm or reschedule requests, and communicate with both students and fellow tutors all within a single, cohesive web application.

1.2 Statement of the Problem

Despite the department's need for accessible peer and tutor support, no dedicated digital platform currently exists to connect Computer Science students at Federal Polytechnic, Ilaro with tutors within their own department. In the absence of such a system, several problems persist:

1. Students have no centralized, searchable directory of available tutors, their subjects, or their areas of specialization (AI, Networking and Cybersecurity, Software Development, and similar options).
2. Scheduling a tutoring session depends on informal, unrecorded communication, which frequently leads to missed appointments, double bookings, and disputes over agreed times.
3. There is no reliable mechanism for conducting the session itself once a time is agreed; students and tutors are left to independently source and coordinate a video-calling tool, which is often unreliable and insecure.
4. Neither party receives timely reminders or notifications when a booking is confirmed, rescheduled, or cancelled, leading to wasted time and missed sessions.
5. There is no structured way for a student to give feedback on a tutor after a session, meaning future students have no basis on which to judge which tutors are most effective.

These problems collectively discourage the informal tutoring culture that already exists within the department from developing into a consistent, trustworthy academic support system. The system described in this report is designed specifically to resolve them.

1.3 Aim and Objectives of the Study

The aim of this study is to design and implement a web-based Tutor and Student E-Learning Meeting System for the Department of Computer Sciences, Federal Polytechnic, Ilaro, that allows students and tutors to discover one another, schedule sessions, and meet live online.

The specific objectives of the study are to:

1. Review existing literature and related systems on e-learning, online tutoring, and real-time video communication technologies.
2. Analyse the current process by which students in the department obtain academic tutoring and identify its shortcomings.
3. Design a relational database and system architecture capable of supporting user registration, tutor discovery, session booking, live video communication, and messaging.
4. Implement the system as a functioning web application using modern, industry-relevant web technologies.
5. Test the system to confirm that it meets its stated functional requirements and performs reliably under normal use.

1.4 Significance of the Study

This study is significant to several groups of stakeholders. To students in the department, it provides a single, reliable channel through which academic support can be requested and received, reducing the informal friction that currently discourages many from seeking help. To tutors, it provides a structured way to manage their availability, confirm sessions on their own terms, and build a visible reputation through student reviews. To the department, it demonstrates a concrete, low-cost model for extending peer-to-peer academic support using tools already familiar to a student population. Finally, to the researcher, the project provides practical, end-to-end experience in full-stack web application design, relational database modelling, and the integration of third-party services such as real-time video communication and transactional email skills directly relevant to a career in software development.

1.5 Scope of the Study

The system is scoped to the Department of Computer Sciences, Federal Polytechnic, Ilaro, and supports two categories of registered users: students and tutors. Its functional scope covers user registration and authentication; tutor profile management, including subject, specialization option, and availability; a searchable tutor directory; a full booking lifecycle (request, confirmation, rejection, cancellation, and rescheduling); live one-on-one video sessions accessed through unique, code-protected links; in-app notifications; downloadable calendar reminders; direct messaging between tutors; a departmental view that groups students by level, study mode, or specialization option; a tutor-submitted student directory; a post-session rating

and review mechanism; and a self-service password recovery flow. The study does not extend to group or class-wide video sessions, in-app payment processing, or integration with the institution's official student records system.

1.6 Limitations of the Study

Several constraints shaped the final system and are acknowledged here rather than presented as oversights. First, user session state is currently managed on the client side rather than through server-issued, cryptographically signed sessions. At the same time, this is adequate for demonstrating the system's functionality, a production deployment intended for continuous public use would warrant a full server-side authentication layer. Second, booking dates and times are stored as plain-text fields rather than as a native date-time database type; this was a deliberate simplification appropriate for a single-timezone deployment, though a larger-scale version of the system would benefit from a proper temporal data type. Third, delivery of transactional email (used for tutor-notification and password-reset messages) depends on a third-party email service whose free tier restricts delivery to a single verified address during testing, which limited full end-to-end email testing across multiple independent accounts. Finally, the study did not include automated unit or integration test suites; verification was instead carried out through structured manual and exploratory testing, described fully in Chapter Four.

1.7 Definition of Terms

1. **E-learning:** the use of electronic technologies, particularly the internet, to access and deliver educational content and instruction outside of a traditional physical classroom.
2. **Tutor:** a registered user of the system, typically a departmental staff member or senior student, who offers academic guidance in a stated subject and specialization.
3. **Booking:** a request made by a student to meet a specific tutor at a proposed date and time, which then moves through a defined status lifecycle.
4. **WebRTC (Web Real-Time Communication):** an open technology standard that enables audio, video, and data to be exchanged directly between web browsers in real time, without the need for a browser plug-in (Suciu et al., 2020).
5. **API (Application Programming Interface):** a defined set of rules that allows one piece of software to request data or services from another.

6. **ORM (Object-Relational Mapping):** a programming technique, and the tool that implements it, that allows a developer to interact with a relational database using the object-oriented constructs of a programming language rather than writing raw structured query language (SQL).
7. **Responsive design:** an approach to web design that ensures a single interface adapts appropriately to different screen sizes, from mobile phones to desktop monitors.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature relevant to the design of the proposed system. It examines the concept and evolution of e-learning, the specific role of online tutoring and scheduling systems, the real-time communication technologies that make live video tutoring possible, the state of ICT adoption within Nigerian tertiary institutions, and the software engineering approaches suited to building a system of this kind. The chapter closes by identifying the specific gap that this project addresses.

2.2 The Concept of E-Learning

E-learning refers to the use of digital technologies and internet connectivity to facilitate teaching and learning outside the constraints of a single physical classroom, allowing learners to access content and interact with instructors from varied locations and at flexible times (Montenegro Rueda et al., 2024). The concept has evolved considerably: early distance education relied on postal correspondence and broadcast media, before the growth of the internet in the 1990s enabled the first learning management systems, and the proliferation of mobile devices in the 2010s made continuous, anywhere access commonplace (Montenegro Rueda et al., 2024).

The literature identifies consistent benefits associated with e-learning, including increased flexibility and accessibility for learners, reduced costs associated with physical materials and travel, and the ability to reach students regardless of geographic location (Montenegro Rueda et al., 2024). At the same time, researchers consistently document real challenges: dependence on stable technical infrastructure, the risk of reduced face-to-face interaction, and the burden placed on both learners and instructors to acquire adequate digital competence before a system can be used effectively (Montenegro Rueda et al., 2024). A systematic review of e-learning effectiveness in higher education similarly concludes that outcomes are strongly conditioned by the quality of a platform's design, the level of instructor engagement it supports, and the technological infrastructure available to its users, rather than by the presence of digital tools alone. These findings directly informed the design priorities of the present project: the system was deliberately kept simple to use, and its core value connecting a student with a tutor and enabling a live conversation was prioritised over any feature that added complexity without a corresponding benefit to either party.

2.3 Online Tutoring and Scheduling Systems

A distinct body of commercial practice has grown around online tutor scheduling. Platforms such as SimplyBook.me, Calendesk, and Appointy allow individual tutors or small tutoring businesses to publish their availability online, accept bookings, and automatically notify both parties of confirmations, reschedules, and cancellations, often bundling a video-conferencing link directly into the confirmation message so that a session can be joined with a single click. These commercial systems demonstrate a set of features that the literature and market practice both treat as essential to a usable booking system: a searchable public listing of providers, self-service scheduling that avoids back-and-forth negotiation, automated reminders, and safeguards against double-booking the same provider for overlapping time slots.

The system developed for this project adopts the same core pattern a searchable tutor directory, a request-and-confirm booking lifecycle, and automated notifications but tailors it specifically to an academic department rather than a commercial tutoring business. Unlike the commercial platforms reviewed, it introduces academically specific context: a tutor's specialization option, a student's level and study mode, and a departmental directory that groups students accordingly, none of which are meaningful concepts in a general-purpose commercial booking tool.

2.4 Real-Time Communication Technologies for Online Learning

Delivering the live video component of an online tutoring session requires a real-time communication technology. WebRTC (Web Real-Time Communication) has emerged as the dominant open, browser-native standard for this purpose, allowing audio, video, and data to be exchanged directly between browsers using JavaScript application programming interfaces, without requiring users to install a plug-in or dedicated application (Suciu et al., 2020). Because it works directly inside a standard web browser, WebRTC removes a major point of friction that would otherwise exist if students and tutors were required to install separate video-calling software before a session could begin (Mohata, Goel, Bahl, & Sengar, 2021).

Several studies have applied WebRTC specifically within educational contexts. Research proposing WebRTC-based collaborative platforms for university e-learning environments has shown that browser-based video, audio, chat, and screen-sharing can be delivered reliably even in networks with limited connectivity, using a lightweight signalling server to coordinate connections between peers. This body of work supports the architectural decision, described fully in Chapter Three, to build the video-session component of this project on an existing WebRTC-based conferencing service (Jitsi Meet) rather than implementing a peer-connection

signalling layer from first principles a choice that concentrates development effort on the scheduling and access-control logic that is unique to this project, while relying on a mature, widely-used implementation for the underlying real-time media transport.

2.5 ICT Adoption in Nigerian Tertiary Institutions

The successful adoption of any digital academic tool in a Nigerian tertiary institution cannot be assumed; it depends on infrastructural readiness, institutional support, and user attitudes. Broader studies of ICT integration in Nigerian tertiary education point to persistent challenges including inconsistent funding for infrastructure, gaps in digital skills among both staff and students, and resistance to changing established, informal ways of working. A study conducted specifically at the Federal Polytechnic, Ilaro found that although there is clear institutional interest in adopting new technology to improve service delivery, this is constrained by staff resistance to change and by gaps in digital competence, and recommended targeted digital-skills exposure through workshops, seminars, and external training as a practical way to close this gap (Obisanya, Aminu, & Adebola, 2025).

This local context has two direct implications for the present project. First, the system's interface was deliberately designed to require no prior technical training: registration, booking, and joining a session all follow patterns already familiar from everyday consumer web and mobile applications. Second, because informal, low-friction adoption is more realistic in this setting than an institution-wide mandate, the system was scoped to a single department rather than the whole institution, allowing it to be introduced and evaluated on a scale where genuine feedback from a real user population is achievable within the timeframe of a final-year project.

2.6 Peer Tutoring in Higher Education

The system's central purpose connecting a student with a more knowledgeable peer or staff tutor sits within a long-established educational practice. Peer tutoring is broadly defined as an arrangement in which academically capable students assist their peers in understanding course content, typically with some form of oversight from academic staff (Topping, 1996). In a widely cited typology and review of the literature, Topping (1996) found substantial evidence that structured peer tutoring produces measurable benefits for both parties: tutees gain from individualized, low-pressure academic support, while tutors themselves consolidate their own understanding through the act of teaching. This dual benefit is one reason the proposed system treats tutors as more than passive service providers features such as a visible average rating and

a dedicated tutor-to-tutor messaging channel were designed to give tutors a stake in, and a community around, the tutoring they provide.

The literature also cautions that the effectiveness of peer tutoring is not automatic: it depends on tutors being genuinely accessible and on the logistics of arranging a session being simple enough not to discourage a hesitant student from asking for help in the first place (Topping, 1996). This finding directly reinforces the problem identified in Chapter One and the central design goal of this project that removing scheduling friction is not a peripheral convenience but a condition for peer tutoring to work at all.

2.7 Trust and Reputation in Digital Platforms

Because the proposed system allows a student to rate a tutor after a session, it was necessary to review how reputation mechanisms function in comparable digital platforms. Early foundational work on reputation systems established that allowing participants in an online exchange to rate one another after an interaction can substitute for the face-to-face trust cues available in physical transactions, giving future users a basis for judging an otherwise unfamiliar counterpart (Resnick, Kuwabara, Zeckhauser, & Friedman, 2000). This principle underpins large-scale commercial platforms and applies equally well to an academic tutoring context: a prospective student choosing between several tutors with similar subjects has no other reliable signal of quality besides the accumulated experience of students who came before them.

Two specific design decisions in the present system follow directly from this literature. First, a review is only permitted from a student who can be verified, at the database level, to have actually completed a confirmed session with the tutor being reviewed preventing the kind of unverified, potentially fraudulent reviews that undermine trust in open reputation systems. Second, a tutor's displayed rating is calculated as a live average across all submitted reviews rather than a single static number, so that it remains an accurate, current reflection of recent student experience.

2.8 Responsive and Mobile-First Web Design

Because students and tutors are expected to access the system from a mix of desktop computers and personal smartphones, the interface had to adapt correctly across screen sizes. Responsive web design is the practice of using flexible grids, flexible images, and conditional styling rules so that a single interface adjusts its layout appropriately to the device rendering it (Marcotte,

2011). A related but distinct discipline, mobile-first design, argues that an interface should be designed beginning with the constraints of the smallest expected screen and then progressively enhanced for larger ones, rather than the reverse, on the grounds that this produces a leaner, more focused experience that still scales up cleanly (Wroblewski, 2011). The interface described in Chapter Four was built following both principles: every page was authored using a utility-first styling approach with breakpoint-specific rules, and navigation elements in particular were designed first for a narrow mobile viewport before being expanded for wider desktop layouts.

2.9 Password Security and Web Application Security Practice

Because the system stores user-chosen passwords, the literature on secure credential storage was also reviewed. Storing a password in plain, readable form is universally discouraged in the security literature, since a single database compromise would then expose every user's real password directly; the accepted alternative is to apply a deliberately slow, salted cryptographic hashing function to each password before storage, so that even a stolen database yields the attacker nothing usable without an infeasible amount of additional computation (Provos & Mazières, 1999). The bcrypt algorithm proposed by Provos and Mazières (1999) remains, more than two decades later, one of the most widely adopted implementations of this principle across web application frameworks, and was adopted for this reason in the present system. More broadly, the system's approach to input validation, server-side enforcement of business rules, and safe handling of user-submitted content was guided by the general web application security practices catalogued by the Open Worldwide Application Security Project (OWASP Foundation, 2021).

2.10 Enabling Technologies

The system was implemented using a modern JavaScript-based full-stack web development toolchain. Next.js, an open-source framework built on top of the React library, was used as the primary application framework. Next.js extends plain React with server-side rendering, file-system-based routing, and the ability to define backend application programming interface (API) endpoints directly within the same project, allowing a single codebase to serve both the user interface and the server-side logic that powers it. PostgreSQL, a mature open-source relational database system, was used for data persistence, accessed through Prisma, an object-relational mapping (ORM) tool that allows the application's data model to be defined once in

a schema file and used consistently across every part of the codebase, while automatically generating and applying the underlying database migrations.

Tailwind CSS, a utility-first styling framework, was used to implement the application's visual design directly within component markup, and the bcrypt hashing algorithm (Provos & Mazières, 1999) was used to store user passwords irreversibly rather than as plain text, following standard secure-authentication practice. Jitsi Meet, an open-source video-conferencing platform built on the W3C WebRTC standard (World Wide Web Consortium, 2021), was integrated to provide the live video session functionality described in Section 2.4, and Resend, a transactional email delivery service, was used to send tutor-notification and password-reset emails. Downloadable session reminders were generated in the iCalendar format defined by the Internet Engineering Task Force (Desruisseaux, 2009), ensuring compatibility with common calendar applications such as Google Calendar, Outlook, and Apple Calendar. Each of these tools is discussed in more technical detail, alongside the specific justification for its selection, in Chapter Four.

2.11 Summary and Identified Gap

The literature reviewed confirms that e-learning delivers genuine benefits when it is well designed and appropriately scoped to its users, that WebRTC is an established and appropriate technology for browser-based live tutoring, and that successful adoption within a Nigerian tertiary institution depends heavily on keeping a system simple and locally relevant rather than technically ambitious. However, the review also confirms that no existing tool commercial or academic combines tutor discovery, a full booking lifecycle, browser-based live video, messaging, and departmental context in a single system scoped specifically to a Nigerian polytechnic computer science department. This is the gap that the Tutor and Student E-Learning Meeting System, presented in the remaining chapters of this report, was designed to fill.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter describes the analysis of the existing (manual) process by which students in the department obtain tutoring, the development methodology adopted for the project, the requirements derived from that analysis, and the resulting design of the proposed system, including its database structure.

3.2 Analysis of the Existing System

As described in Chapter One, the department currently has no dedicated system for connecting students with tutors. In its place is an informal process: a student who requires help typically asks classmates or a course representative for a recommendation, contacts a prospective tutor directly (commonly through a personal phone number or social media account), and negotiates a time and location by ordinary conversation. If a video call is required, the two parties independently agree on whichever consumer video-calling application both happen to have installed.

This existing process has several clear weaknesses that directly motivate the requirements of the proposed system. It has no central record of who is available to tutor which subject, so a student's ability to find help depends entirely on their existing social network. It has no defined confirmation step, so a tutor's silence can easily be mistaken for agreement. It provides no reminder mechanism, so agreed sessions are frequently forgotten. And it leaves no record from which a future student could learn whether a given tutor's sessions were actually useful.

3.3 Methodology

The system was developed using an iterative, agile-inspired methodology, rather than a single up-front design followed by one long implementation phase. Agile approaches to software development, formally articulated in the Manifesto for Agile Software Development (Beck et al., 2001), favour short cycles of building, testing, and refining a working feature over exhaustive advance planning, allowing requirements to be adjusted in response to what is learned from each cycle rather than being fixed prematurely. In practice, this meant that each major feature of the system user registration, the tutor directory, the booking lifecycle, live video sessions, notifications, messaging, and the rating system was designed, implemented, and manually tested as a discrete increment before the next was started, with earlier increments

revisited and refined once later features revealed a need to do so (for example, the booking data model was extended twice, first to support a unique video-room identifier and later to support a tutor-confirmed session access code, once the video-session and security requirements became concrete).

3.4 Requirement Specification

3.4.1 Functional Requirements

1. The system shall allow a new user to register as either a student or a tutor, capturing role-appropriate information (for students: matriculation number, level, study mode, and specialization option where applicable; for tutors: subject and specialization).
2. The system shall allow a registered user to log in and log out, and to recover access to their account through a self-service password reset flow.
3. The system shall present a searchable, filterable directory of tutors, showing each tutor's subject, specialization, and rating.
4. The system shall allow a student to request a booking with a specific tutor for a chosen date and time, and shall prevent a new booking from being created against a tutor who already has a confirmed session at that exact date and time.
5. The system shall allow a tutor to confirm, reject, cancel, or propose a new date and time for a booking, and shall notify the affected student of each change.
6. The system shall generate a unique, permanent video-room identifier for every booking and every ad-hoc instant session, and shall generate a short numeric access code that a student must supply before joining a booking-linked session.
7. The system shall allow a confirmed booking to be exported as a downloadable calendar file with an embedded one-hour-before reminder.
8. The system shall allow a tutor to toggle their own availability, and shall prevent students from booking a tutor who is currently marked unavailable.
9. The system shall allow registered tutors to exchange direct messages with one another.
10. The system shall allow a student who has completed a confirmed session with a tutor to submit exactly one numeric rating and optional comment for that tutor, and shall display a live-calculated average rating on the tutor's profile.
11. The system shall present each student with a departmental view of their own academic details and of other students sharing their level, study mode, or specialization option.

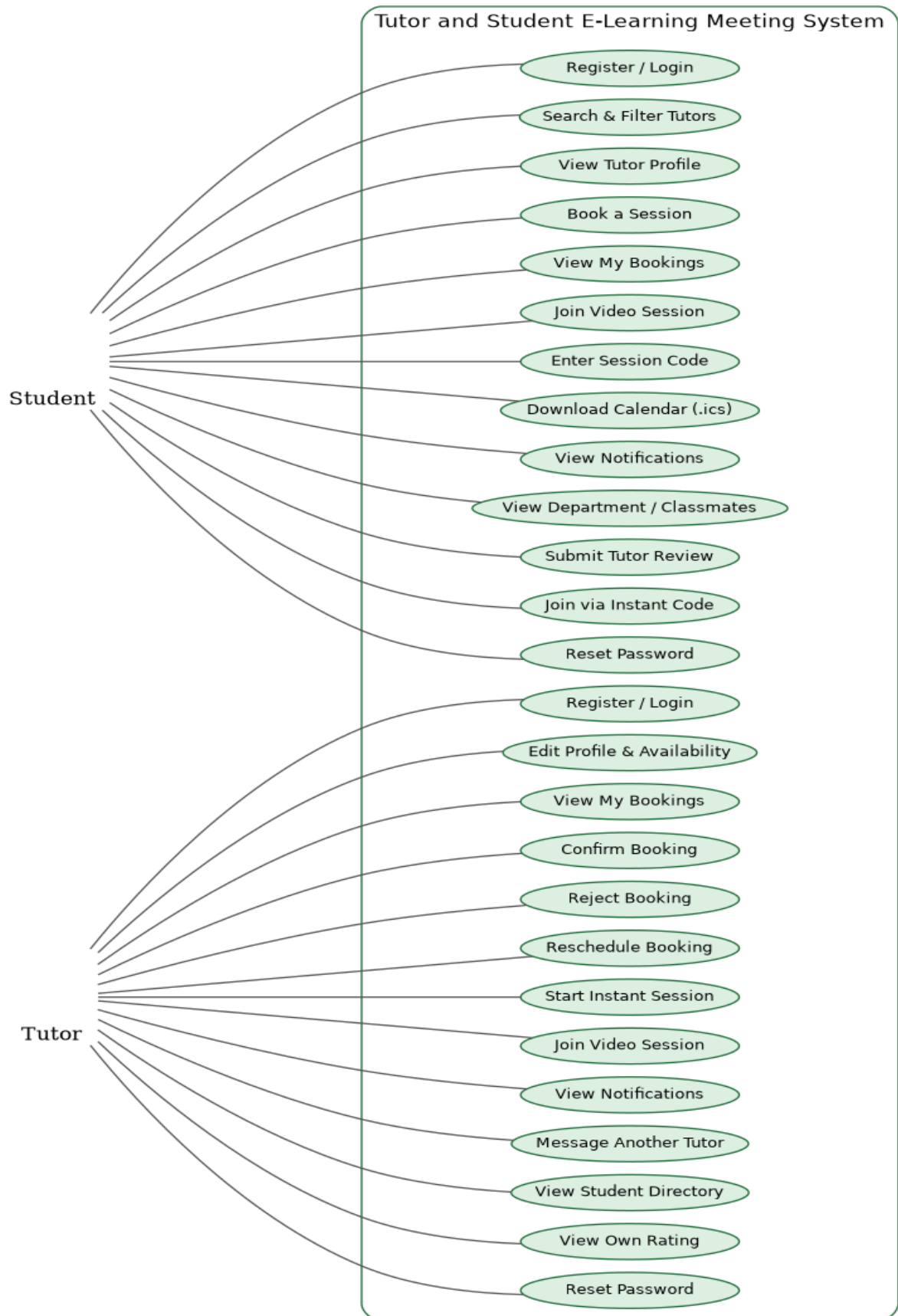


Figure 3.1: Use Case Diagram of the Tutor and Student E-Learning Meeting System

3.4.2 Non-Functional Requirements

1. Usability: the interface shall be operable by a general student and tutor population without prior training, consistent with the ICT-adoption considerations discussed in Chapter Two.
2. Security: user passwords shall never be stored in plain text; they shall be irreversibly hashed before being persisted.
3. Responsiveness: the interface shall adapt correctly to both desktop and mobile screen sizes.
4. Reliability: booking status transitions and video-room/code generation shall be enforced on the server side, not solely in the browser, so that they cannot be bypassed.
5. Performance: pages that display frequently changing data, such as notifications and messages, shall refresh automatically at short, regular intervals without requiring a manual page reload.

3.5 Database Design

The system's data is stored in a relational PostgreSQL database, structured according to the relational model of data first formalised by Codd (1970), in which information is organised into named tables of rows and columns and relationships between records are expressed through shared key values rather than physical storage links (Elmasri & Navathe, 2016). The design was structured around five core entities. The User entity stores every registered account, distinguished by a role field into student and tutor records, and holds role-relevant attributes including name, email, hashed password, and for students matriculation number, level, study mode, and specialization option, and for tutors subject, specialization option, biography, availability flag, and rating. The Booking entity represents a single tutoring request and its full lifecycle, holding references to the student and tutor involved, the proposed subject, date and time, a status field (pending, confirmed, or cancelled), a unique video-room identifier, a session access code generated once a booking is confirmed, and flags indicating whether the student has been notified of the current state and whether the booking represents a tutor-proposed reschedule.

The QuickSession entity supports ad-hoc video sessions that are not tied to a specific booking, allowing a tutor to generate a session code and share it through any channel of their choosing. The Message entity stores individual messages exchanged between two tutors, including sender and receiver identifiers, message content, and a read flag. The Review entity stores a single rating and optional comment submitted by a student for a tutor, with a database-level uniqueness constraint across the student-tutor pair ensuring that a student can rate any one tutor

only once. Figure 3.2 summarises the relationships between these entities: a User with role student initiates many Bookings and may submit many Reviews (one per distinct tutor); a User with role tutor receives many Bookings, may hold many Reviews, and exchanges many Messages with other tutor Users.

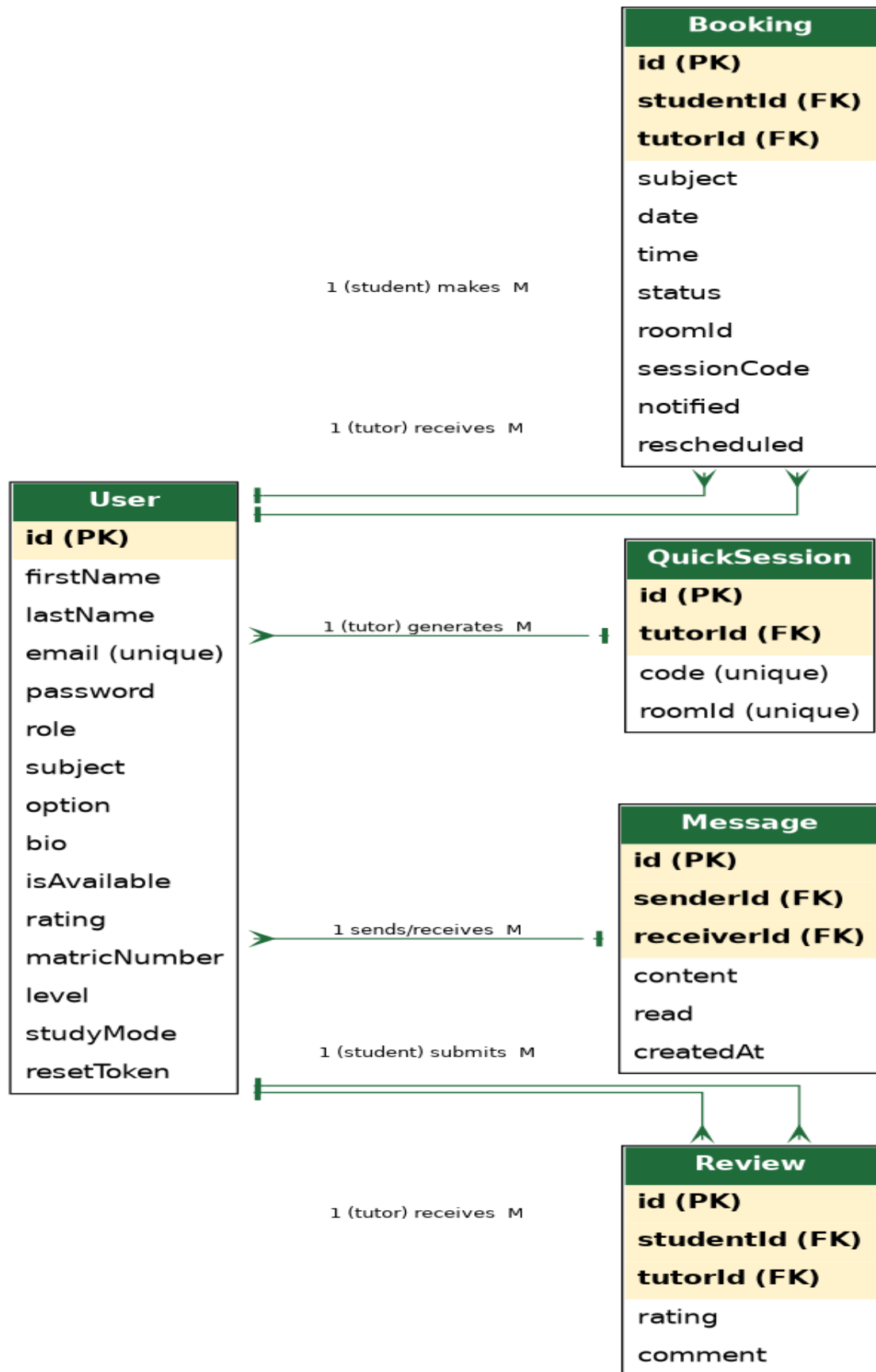


Figure 3.2: Entity-Relationship Diagram of the System Database

3.6 System Architecture

The system follows a full-stack, single-codebase architecture made possible by the Next.js framework (Vercel Inc., 2026). The browser-facing client is built from React components and communicates with a set of server-side API routes defined within the same project an architectural pattern consistent with the representational state transfer style described by Fielding (2000), in which resources (bookings, messages, reviews, and so on) are manipulated through a uniform set of stateless requests. These routes, in turn, communicate with the PostgreSQL database through the Prisma ORM. Live video sessions are handled by embedding the Jitsi Meet WebRTC client directly into a dedicated session page, with the system's own database supplying only the unique room identifier and the access-control logic that determines whether a given user is permitted to join at a given time the actual audio and video transport is delegated entirely to the underlying WebRTC infrastructure, consistent with the architectural rationale discussed in Section 2.4. Outbound transactional email (tutor notifications and password-reset messages) is dispatched through the Resend email application programming interface, called from the relevant server-side API route rather than from the browser.

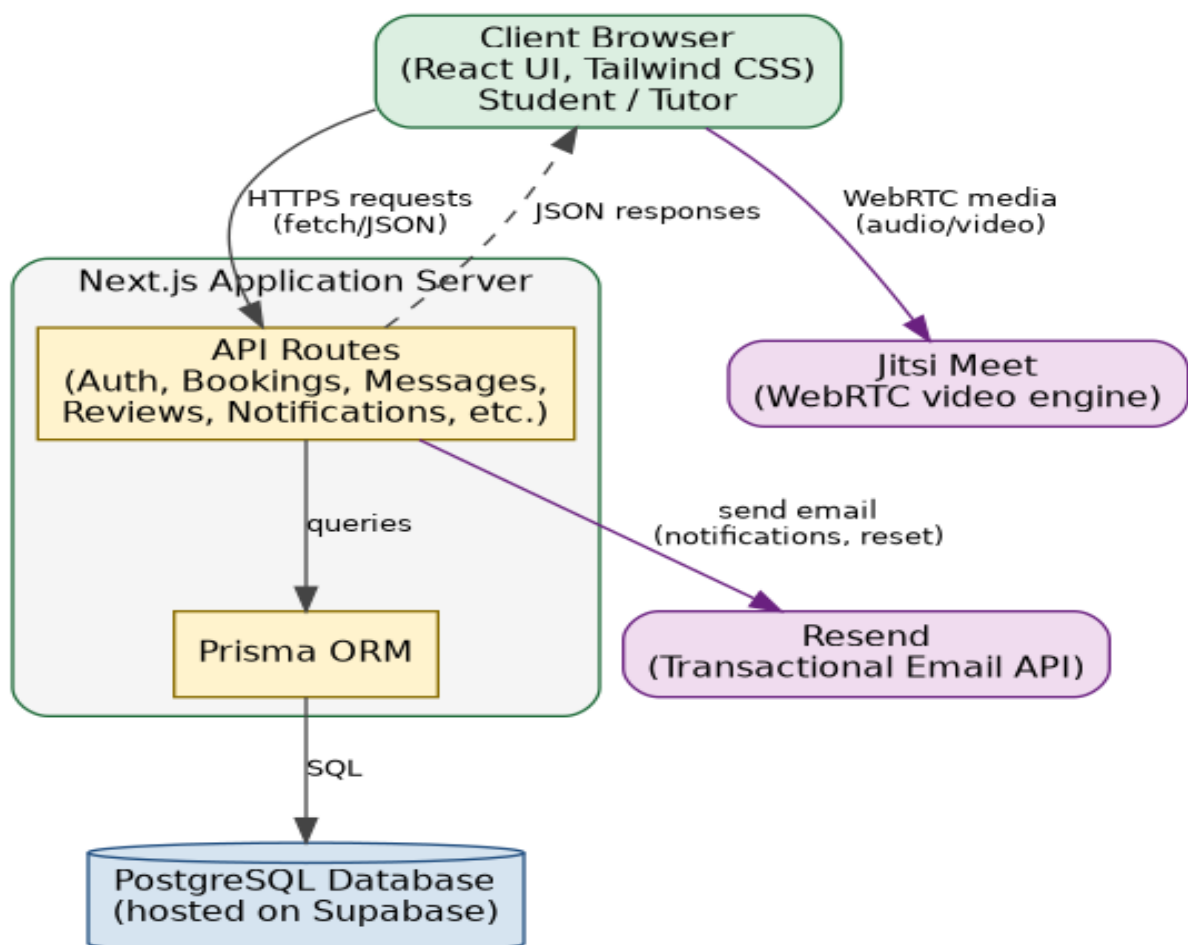


Figure 3.3: System Architecture Diagram

3.7 Input and Output Design

Input to the system is collected through standard web forms: a role-aware registration form, a login form, a booking-request form (date and time selection), a review form (a five-point star selector with an optional comment field), and a message-composition field. Where an input has a defined valid format for example, a ten-digit matriculation number, or a password meeting a minimum-strength policy validation is enforced both in the browser, to give the user immediate feedback, and again on the server, so that the rule cannot be bypassed by a request sent directly to the API. Output is presented through role-aware dashboards (a distinct home view for tutors and for students), a searchable tutor directory, a bookings list showing each booking's current status, an in-app notification list, a downloadable calendar file, and a messaging inbox and conversation thread view.

CHAPTER 4

SYSTEM IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter describes how the design presented in Chapter Three was implemented, the specific tools and technologies used and the justification for each, the system's minimum operating requirements, and the testing carried out to verify that the completed system meets its stated requirements.

4.2 Choice of Tools and Technologies

Next.js (built on React) was selected as the application framework because it allows a single project to define both the browser-facing interface and the server-side API endpoints that back it, removing the need to build and separately deploy a distinct backend service an important practical consideration for a project developed and maintained by a single developer. PostgreSQL was selected as the database engine for its maturity, strong support for relational integrity constraints (used, for example, to enforce the one-review-per-tutor-per-student rule described in Chapter Three), and its availability as a managed, hosted service through Supabase, which removed the need to administer database infrastructure directly. Prisma was chosen as the object-relational mapping layer because it allows the entire data model to be defined once, in a single schema file, from which it generates both the database migration scripts and a fully type-checked client used throughout the application code, reducing the risk of a mismatch between the code and the actual database structure.

Tailwind CSS was used for styling because its utility-first approach allowed the interface's visual design including the animated, gradient-themed elements described later in this chapter to be implemented quickly and consistently without maintaining a large separate stylesheet. The bcrypt library was used to hash user passwords before storage, following standard practice for secure authentication. Jitsi Meet was integrated as the video-conferencing engine because it is open-source, requires no per-session licensing cost, and exposes a simple external application programming interface that could be embedded directly into a page while the system's own database controlled who was permitted to reach that page. Resend was used to deliver transactional email because it offers a straightforward API suited to a small-scale project, in preference to configuring a general-purpose mail server.

4.3 System Requirements

4.3.1 Hardware Requirements

1. A computer or mobile device capable of running a modern web browser.
2. A stable internet connection, particularly for live video sessions.
3. At least 4GB of RAM and a dual-core processor for a smooth development and testing experience.

4.3.2 Software Requirements

1. Node.js runtime environment (for development and running the server).
2. A modern web browser supporting WebRTC (for example, Google Chrome, Mozilla Firefox, or Microsoft Edge).
3. A PostgreSQL-compatible database instance (hosted via Supabase in this implementation).
4. Git and a code editor, for development.

4.4 System Implementation

The completed system is organised into a small number of cohesive modules, each corresponding to one of the functional requirements identified in Chapter Three.

The Authentication module handles registration and login for both roles, applies role-specific validation (including the ten-digit matriculation-number format and password-strength rule described in Chapter Three), hashes passwords with bcrypt before storage, and supports self-service password recovery through a time-limited, single-use reset token delivered by email.

The Tutor Directory and Profile module presents a searchable, filterable list of tutors and an individual profile page for each, showing subject, specialization, biography, live-calculated average rating, and current availability; a tutor's own profile page additionally allows in-place editing of these details and toggling of the availability flag.

The Booking module implements the full request-confirm-reject-reschedule-cancel lifecycle described in Chapter Three. A booking is created in a pending state; a tutor may confirm it, which generates a session access code and returns it to the tutor to share with the student, or reject it, which marks it cancelled and notifies the student; a cancelled booking may subsequently be rescheduled by the tutor, which updates its date and time and returns it to a pending state with the student notified afresh. Server-side logic rejects any new booking request that would conflict with a tutor's existing confirmed session at the same date and time.

The Live Video Session module generates a unique room identifier for every booking at the moment it is created, and a separate mechanism for ad-hoc, booking-independent instant sessions generated on demand by a tutor. Access to a booking-linked session is gated on the client interface by two conditions the booking must be confirmed, and the current time must be within fifteen minutes of the scheduled time after which the student is prompted to enter the access code shared by the tutor before being admitted to the Jitsi Meet room identified by the stored room identifier.

The Notification and Calendar module surfaces booking-status changes (confirmation, cancellation, and reschedule) to the affected student through an in-app notification list, capped to the ten most recent items and marked as read individually rather than all at once, and generates a downloadable calendar file (in the standard .ics format) for any confirmed booking, with a reminder alarm embedded one hour before the scheduled start time.

The Messaging module allows any tutor to open a conversation with any other tutor directly from that tutor's profile page, presenting an inbox of active conversations and a chat-style thread view that refreshes automatically to show new incoming messages without a manual reload.

The Review and Rating module allows a student to submit a single rating (accompanied by an optional written comment) for any tutor with whom they have completed a confirmed session, enforced by a database-level constraint preventing a second submission for the same tutor; the tutor's displayed rating is calculated live as the average of all submitted reviews.

The Departmental View module presents each student with a summary of their own academic details (matriculation number, level, study mode, and specialization option where applicable) alongside a list of other students who share the same grouping, while a corresponding tutor-facing module presents the full roster of registered students, sorted alphabetically by surname.

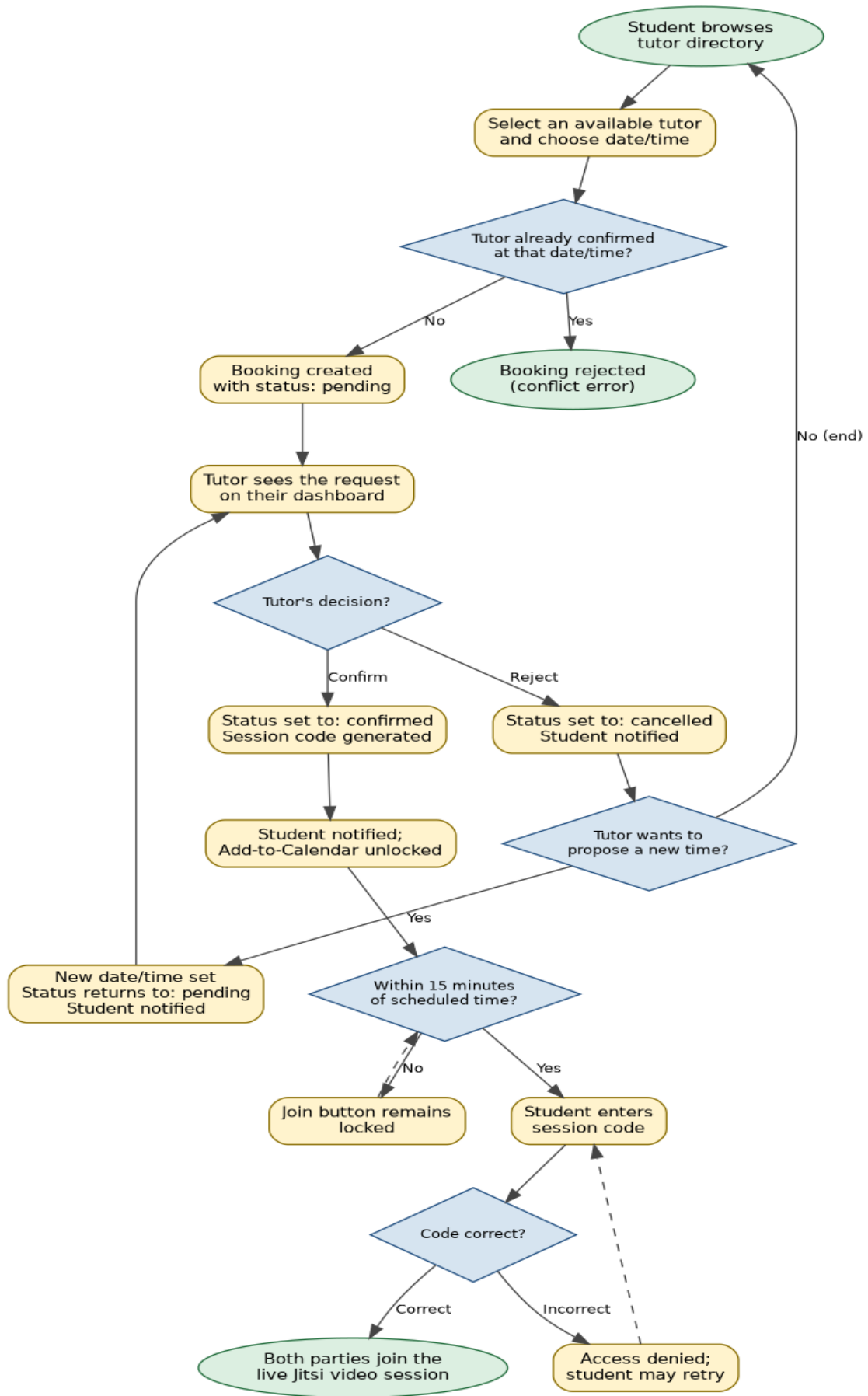


Figure 4.1: Flowchart of the Booking and Live Session Process

4.5 System Testing

Testing was carried out throughout development rather than as a single final phase, following the iterative methodology described in Chapter Three: each module was tested as soon as it was implemented, and again whenever a later change touched the same code. Two complementary testing techniques were applied, following standard software testing practice (Myers, Sandler, & Badgett, 2011). Black-box testing was used to verify each feature strictly from the perspective of its expected external behaviour for example, confirming that submitting a booking request with a date in the past correctly produced an error message, without reference to how that validation was implemented internally. White-box testing was applied where a feature's internal logic carried a specific risk of an incorrect edge case, most notably the booking-conflict check, the review uniqueness constraint, and the time-based video-session access gate, where the underlying conditional logic was inspected directly to confirm every branch behaved as intended.

Table 4.1 summarises a representative sample of the test cases executed against the completed system.

Table 4.1: Sample Test Cases and Results

Feature	Test Case	Expected Result	Result
Registration	Register a student with a matriculation number shorter than 10 digits	Registration blocked with a validation error	As expected
Booking	Student requests a booking with a tutor already confirmed at that date/time	Request rejected with a conflict error	As expected
Booking lifecycle	Tutor rejects a pending booking	Booking status becomes cancelled; student notified	As expected
Video session	Student attempts to join a confirmed booking more than 15 minutes before its scheduled time	Join button remains locked	As expected
Video session	Student enters an incorrect access code	Access denied with an error message; correct code succeeds	As expected

Reviews	Student submits a second review for the same tutor	Submission blocked by uniqueness constraint	As expected
Password reset	User requests a reset link, then submits a new password not meeting the strength policy	Reset rejected with a validation error; compliant password succeeds	As expected

In every case listed above, the observed result matched the expected result once the corresponding feature had been fully implemented; issues discovered during testing (for example, a Prisma client staleness error following a schema change, and a case where a database migration was not created before the application code that depended on it) were resolved by regenerating the Prisma client and, respectively, by explicitly re-running the migration command, after which the affected feature passed re-testing.

The interface's usability was guided throughout by long-established human-computer interaction principles: that a well-designed system should require no specialist training and should communicate its state clearly at every step (Nielsen, 1994), and that a user should never have to stop and think about how to accomplish a task because the correct action is always self-evident (Krug, 2014). These principles directly motivated decisions described earlier in this chapter, such as gating the video-join button visibly rather than silently, and surfacing a tutor's live-calculated rating rather than requiring a student to look it up separately.

4.6 System Interface

This section presents screenshots of the completed system's user interface, illustrating the modules described in Section 4.4 as they appear to an actual user.

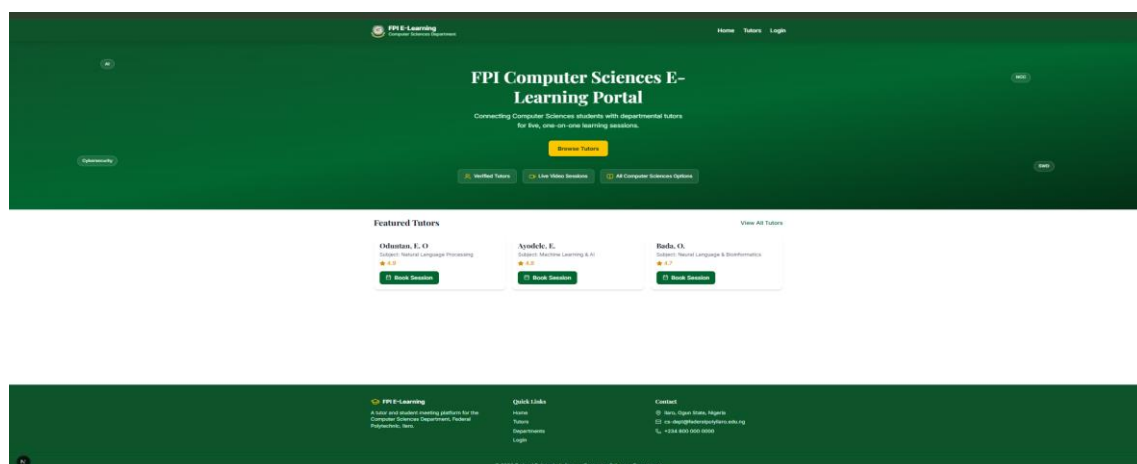


Figure 4.2: Homepage, showing the hero section and featured tutors

Figure 4.3: Login page with role selection (Student/Tutor)

Figure 4.4: Student registration form, showing role-specific fields (matriculation number, level, study mode) and live password-strength validation

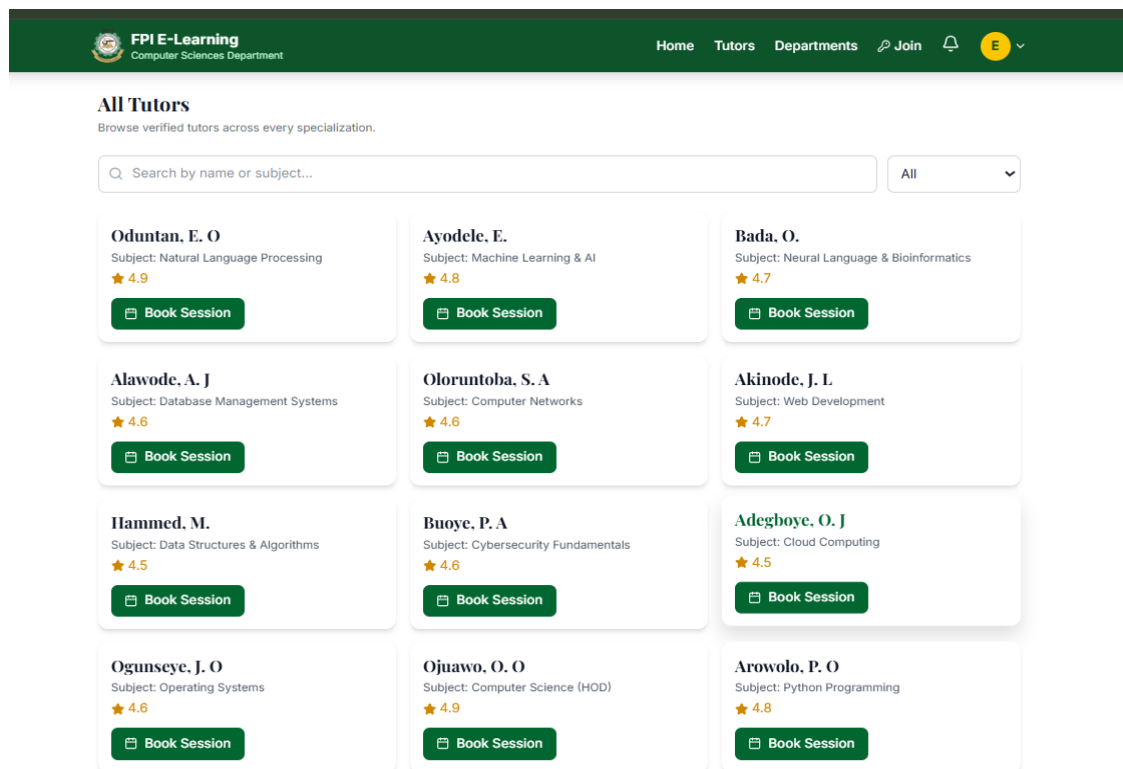


Figure 4.5: Tutor directory, showing search, filter, and specialization-based browsing

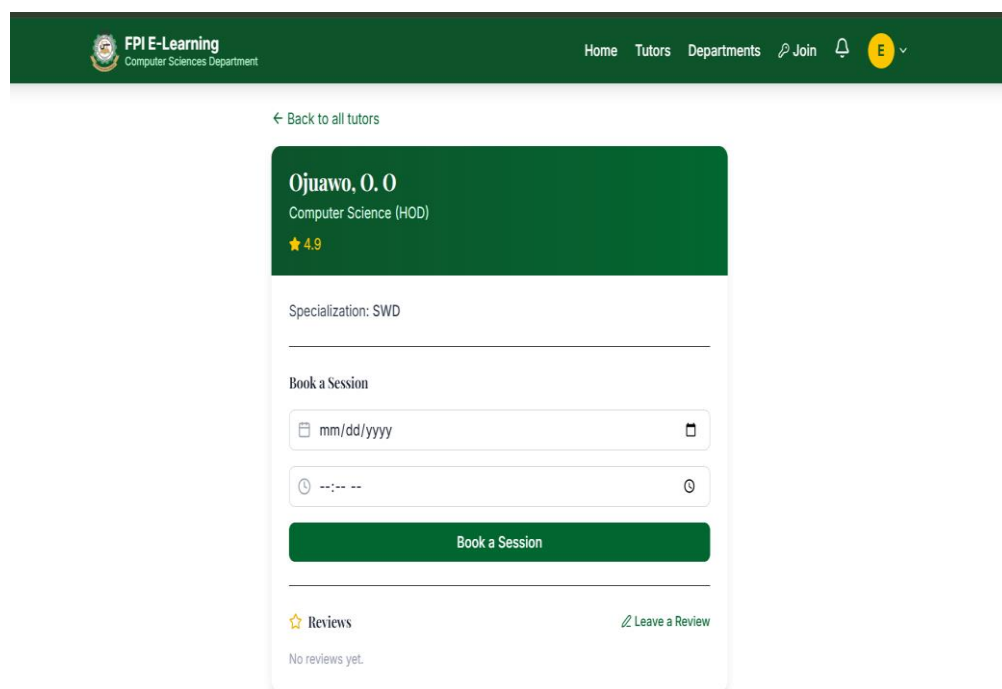


Figure 4.6: Individual tutor profile, showing the booking form and reviews section

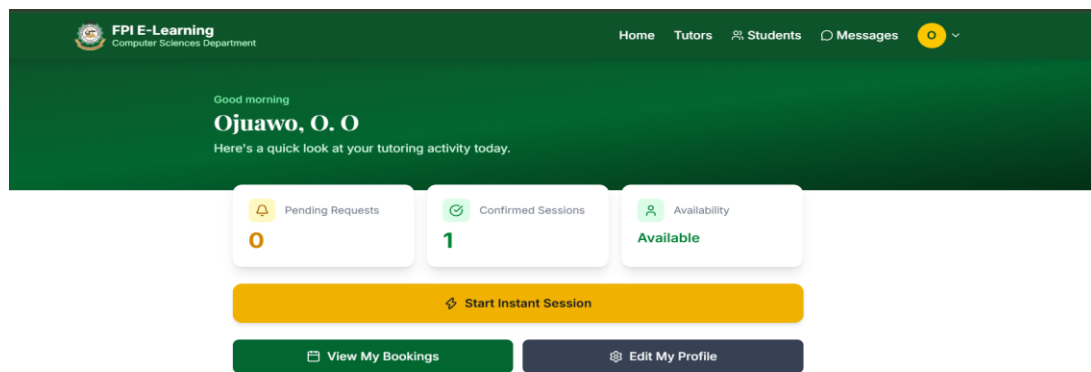


Figure 4.7: Tutor home dashboard, showing pending requests, confirmed sessions, availability status, and the instant-session action

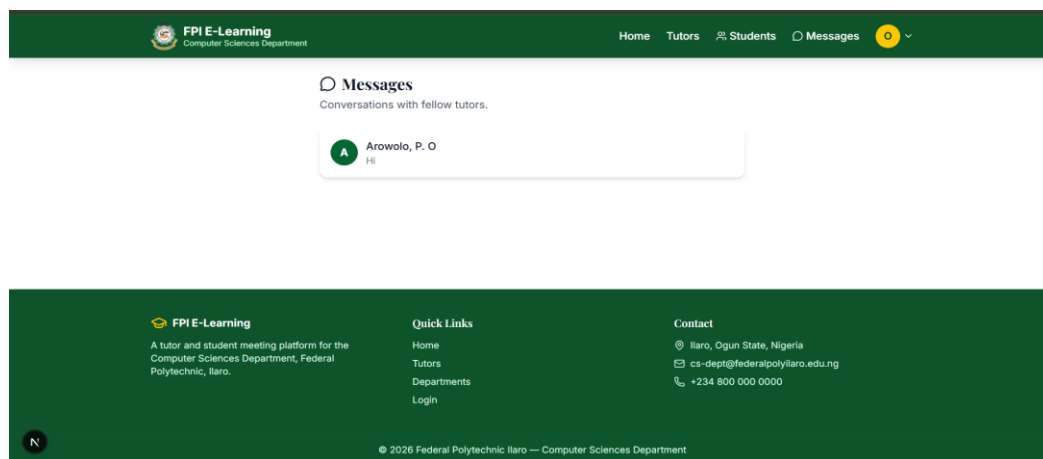


Figure 4.8: Messaging inbox, listing active conversations between tutors

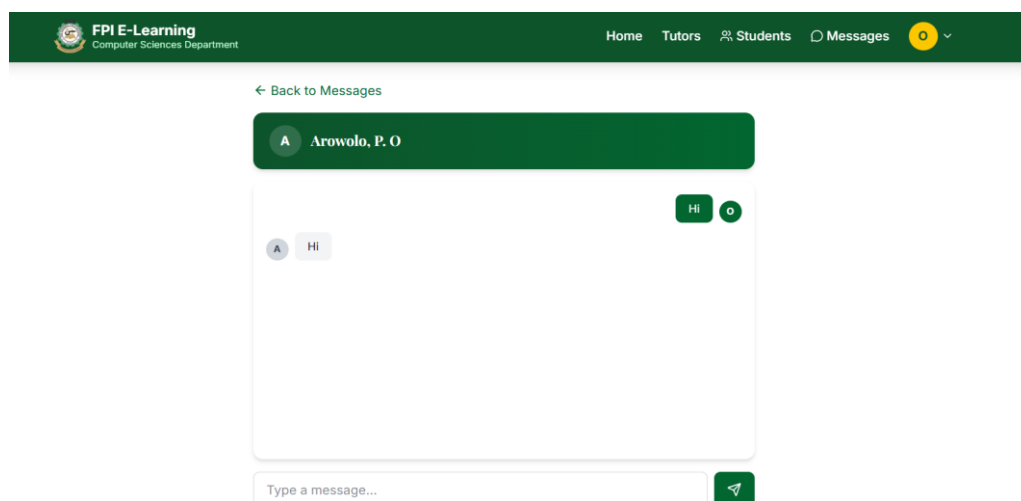


Figure 4.9: Message thread view between two tutors

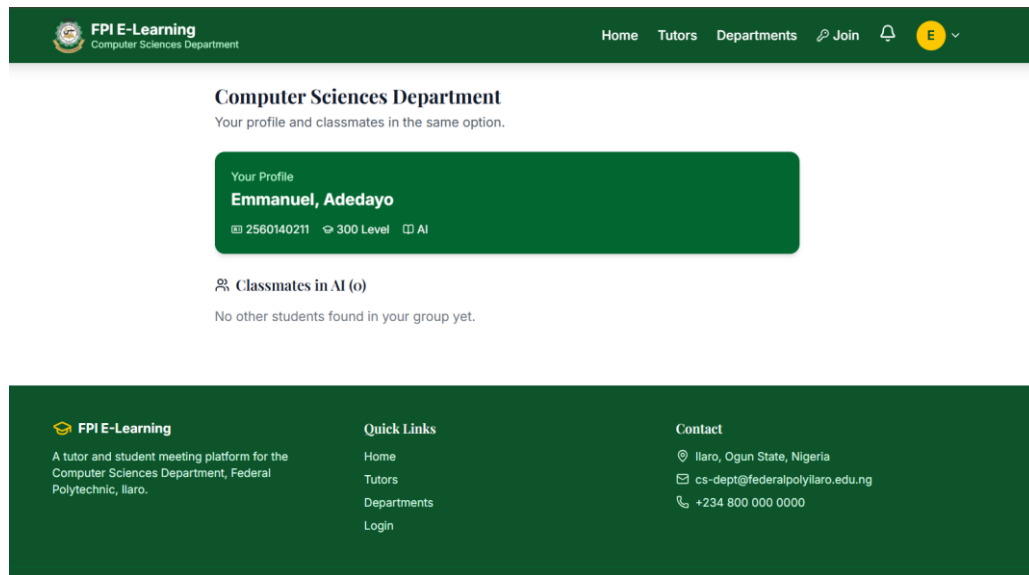


Figure 4.10: Departmental view, showing a student's profile summary and classmates grouped by specialization option

4.7 System Documentation

The system requires no installation by its end users, being accessed entirely through a standard web browser. To use the system, a new user visits the application's landing page and selects either the Student or Tutor tab on the sign-up form, supplies the requested details, and creates an account. A student may then browse the tutor directory, open a tutor's profile, and submit a booking request specifying a preferred date and time. A tutor, upon logging in, is directed to a dedicated tutor home page summarising pending requests and confirmed sessions, from which bookings can be confirmed, rejected, or rescheduled. Once a booking is confirmed, the tutor is shown a one-time access code to share with the student through any channel of their choosing; the student uses this code to unlock the live video session once the scheduled time is near. Full operational guidance for each feature is embedded contextually within the interface itself, consistent with the usability requirement identified in Chapter Three.

CHAPTER 5

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This study set out to design and implement a web-based Tutor and Student E-Learning Meeting System for the Department of Computer Sciences, Federal Polytechnic, Ilaro, in response to the absence of any centralized channel through which students in the department could discover, book, and meet with academic tutors. Chapter One established the background and scope of the problem; Chapter Two reviewed literature on e-learning, online tutoring platforms, real-time browser-based video communication, and the specific conditions affecting ICT adoption within Nigerian tertiary institutions, and identified a clear gap that no existing system addresses. Chapter Three translated the findings of that review into a concrete set of functional and non-functional requirements and a relational database and system architecture capable of meeting them. Chapter Four described the resulting implementation covering authentication, tutor discovery, the full booking lifecycle, live video sessions gated by time and access code, notifications and calendar reminders, tutor-to-tutor messaging, a student rating and review system, and a departmental directory view and the testing carried out to verify each feature against its requirements.

5.2 Conclusion

The completed system successfully demonstrates that a single, cohesive web application can replace the fragmented, informal process by which students in the department currently seek academic tutoring. By combining tutor discovery, structured scheduling, secure live video communication, and post-session feedback within one platform, the system directly addresses each of the specific shortcomings identified in Chapter One: it gives students a searchable record of available tutors, replaces ad-hoc verbal agreements with a defined and auditable booking lifecycle, provides a reliable and access-controlled channel through which the tutoring session itself can take place, and gives future students a basis through submitted ratings for judging which tutors are likely to be most helpful. The project further demonstrates that a modern, JavaScript-based full-stack toolchain (Next.js, PostgreSQL, and Prisma) is well suited to building a system of this scope within the practical time and resource constraints of a final-year undergraduate project.

5.3 Recommendations

The Department of Computer Sciences should consider piloting the system with a small cohort of registered tutors and students to gather real usage feedback before any wider rollout.

Tutors who volunteer to use the system should be given brief orientation on the confirmation and access-code workflow, since the value of the security code depends on it being shared correctly with the intended student.

Should the institution wish to extend the system beyond the Department of Computer Sciences, the specialization-option field should be generalised so that it can represent the course offerings of other departments.

Future maintainers of the system should prioritise migrating user authentication from client-side storage to a server-issued session mechanism before any deployment intended for continuous, unsupervised public use.

5.4 Suggestions for Further Work

A number of enhancements were deliberately left outside the scope of this project and are recommended as directions for further work. First, the current client-side authentication approach should be replaced with a proper server-side session or token-based authentication mechanism, providing stronger protection against unauthorized access as the system is deployed to a wider audience. Second, the storage of booking dates and times as plain text fields should be migrated to a native date-time database type, which would allow more robust timezone handling should the system ever be used across multiple locations. Third, an automated test suite (covering unit and integration testing) should be introduced to complement the manual testing described in Chapter Four, reducing the risk of regressions as the system continues to evolve. Finally, future work could extend the system to support group tutoring sessions involving more than two participants, integrate directly with the institution's official student records system to automatically verify matriculation numbers at registration, and introduce an in-app payment mechanism should the department wish to support paid, structured tutoring in addition to the current voluntary model.

REFERENCES

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. <https://agilemanifesto.org/>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Desruisseaux, B. (2009). Internet calendaring and scheduling core object specification (iCalendar) (RFC 5545). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5545>
- Ecma International. (2024). ECMAScript 2024 language specification (15th ed.). <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].
- Jitsi (8x8, Inc.). (2024). Jitsi Meet handbook. <https://jitsi.github.io/handbook/>
- Krug, S. (2014). *Don't make me think, revisited: A common sense approach to web usability* (3rd ed.). New Riders.
- Marcotte, E. (2011). *Responsive web design. A Book Apart*.
- Mohata, R. K., Goel, A., Bahl, V., & Sengar, N. (2021). Peer to peer real-time communication using WebRTC. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 7(6), 125–130. <https://doi.org/10.32628/CSEIT217647>

- Montenegro Rueda, M., Fernández Cerero, J., Fernández Cerero, D., & López Meneses, E. (2024). Perspectives on online learning: Advantages and challenges in higher education. *Contemporary Educational Technology*, 16(4), ep525. <https://doi.org/10.30935/cedtech/15011>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). Wiley.
- Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.
- Obisanya, A. R., Aminu, T. N., & Adebola, O. L. (2025). Technology adoption and service delivery in Federal Polytechnic Ilaro, Ogun State. *Lead City Journal of the Social Sciences*, 10(2), 72–84.
- OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. <https://owasp.org/Top10/>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Prisma Data, Inc. (2024). Prisma ORM documentation. <https://www.prisma.io/docs>
- Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference, FREENIX Track* (pp. 81–91). USENIX Association.
- Resend, Inc. (2024). Resend documentation. <https://resend.com/docs>
- Resnick, P., Kuwabara, K., Zeckhauser, R., & Friedman, E. (2000). Reputation systems. *Communications of the ACM*, 43(12), 45–48. <https://doi.org/10.1145/355112.355122>
- Rogers, E. M. (2003). *Diffusion of innovations* (5th ed.). Free Press.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

- Suciu, G., Stanciu, A., Ijaz, M., Zamfir, M., & Militaru, T. G. (2020). WebRTC role in real-time communication and video conferencing. In Proceedings of the 4th International Symposium on Computer Science and Intelligent Control. IEEE.
- Tailwind Labs Inc. (2024). Tailwind CSS documentation. <https://tailwindcss.com/docs>
- Topping, K. J. (1996). The effectiveness of peer tutoring in further and higher education: A typology and review of the literature. Higher Education, 32(3), 321–345. <https://doi.org/10.1007/BF00138870>
- Vercel Inc. (2026). Next.js documentation. <https://nextjs.org/docs>
- Wroblewski, L. (2011). Mobile first. A Book Apart.
- World Wide Web Consortium. (2021). WebRTC 1.0: Real-time communication between browsers (W3C Recommendation). <https://www.w3.org/TR/webrtc/>